

2026 年度高 2 情報

数値計算の誤差と システム開発における仕様定義

2026/09/12

数値計算における誤差の分類

コンピュータにおける実数表現の限界

有限のビット数で実数を表現するため、数学的な真値とコンピュータ上の表現値の間に誤差が生じる。

- **丸め誤差 (Rounding Error)**

無限小数を有限の桁数で打ち切る際に生じる誤差。

例: $0.1_{(10)}$ は2進数では循環小数となり、厳密に表現できない。

- **桁落ち (Loss of Significance)**

値がほぼ等しい2つの数値の差を求めた際、有効数字が極端に減少する現象。

- **オーバーフロー (Overflow)**

割り当てられたデータ型の表現可能な最大値を超える現象。

桁落ち (Loss of Significance) とは

情報（精度）の消失現象

- 値が極めて近い2つの数値の差を計算すると、上位の有効数字が打ち消し合い、精度が著しく低下する現象。
- 例：有効数字5桁の計算機で $1.0001 - 1.0000$ を計算すると、結果は 0.0001 となり、有効数字が1桁に減少してしまう。
- 1学期に扱った「オーバーフロー（桁あふれ）」は表現できる最大値の超過によるエラーだが、「桁落ち」は計算過程で静かに精度が失われる現象である。

2次方程式における桁落ちの具体例

標準的な解の公式による計算の破綻

- 方程式 $x^2 + 10^{15}x + 10^{14} = 0$ の解を求める。
- 解の公式 $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ を適用する。
- $b = 10^{15}$ と極めて大きいため、 $\sqrt{b^2 - 4ac} \approx 10^{15}$ となる。
- 大きい方の解を求める際、分子が $(-10^{15} + 10^{15})$ のような近い値同士の減算となり、桁落ちが発生する。
- **実際の出力例**（倍精度浮動小数点数）：

$$\frac{-b + \sqrt{b^2 - 4ac}}{2a} = -0.125$$

アルゴリズムの工夫による桁落ちの回避 (1)

分子の有理化を利用した数式の変形

- 桁落ちの原因となる「近い値同士の減算」を避けるため、数学的な式変形（分子の有理化）を行う。
- 解の公式の分子と分母に $(-b - \sqrt{b^2 - 4ac})$ を掛けると、以下のように変形できる。

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a} = \frac{2c}{-b - \sqrt{b^2 - 4ac}}$$

- これにより、計算途中の厄介な引き算が「同符号同士の加算」に変わり、情報（精度）の消失を防ぐことができる。

アルゴリズムの工夫による桁落ちの回避 (2)

実際の計算結果の比較

- 先ほどの $b = 10^{15}, c = 10^{14}$ を変形した式に適用する。
- 分母は $(-10^{15} - 10^{15})$ のような計算となり、近い値の減算が完全に回避される。
- **実際の出力例**（倍精度浮動小数点数）：

$$\frac{2c}{-b - \sqrt{b^2 - 4ac}} = -0.1000000000000002$$

- **結論:** 計算機の誤差は、数学的なアプローチによって克服できる。工夫ひとつで真値（約 -0.1 ）を正確に導き出せる。

数値計算の誤差に起因するシステム障害

パトリオットミサイル迎撃失敗（1991 年）

- **原因:** 時間計測における丸め誤差の蓄積。
- システムの内部時計（0.1 秒単位）を 2 進数で処理する際、打ち切りによる丸め誤差が発生。稼働時間の経過とともに誤差が増大し、目標座標の計算に致命的なズレを生じさせた。

アリアン5 型ロケット爆発事故（1996 年）

- **原因:** 型変換に伴うオーバーフロー。
- 水平速度のデータを「64 ビット浮動小数点数」から「16 ビット符号付き整数」へ変換（キャスト）する際、値が 16 ビット整数の上限（32767）を超過し、システムがダウンした。

モジュール間の不整合に起因するシステム障害

ハードウェアや計算アルゴリズムの限界だけでなく、ソフトウェア開発におけるモジュール間の連携の不備も障害を引き起こしうる。

マーズ・クライメイト・オービター通信途絶（1999年）

- **原因:** ソフトウェア間での単位系の不一致。
- 探査機の姿勢制御システム（開発: ロッキード・マーティン社）は推力を**ヤード・ポンド法**（ポンド重）で計算し、出力した。
- 一方、軌道計算システム（開発: NASA）は入力データを**SI単位系**（ニュートン）として処理した。
- プログラム自体に構文エラーはなかったが、モジュール間のインターフェース仕様（単位）の定義・共有が欠落していたため、誤った軌道修正が行われた。

ソフトウェア開発におけるプロジェクトマネジメント

- 前述の NASA の事例が示すように、複数人でシステムを開発する際、各々が独立してコードを記述し、最後に結合する手法は破綻を招く。
- 個人単位のプログラミング技術に加え、チーム全体の仕様を統制し、開発プロセスを管理する**プロジェクトマネジメント (PM)**が不可欠である。
- 特に、リソース（開発時間や技術力）が限られている学校でのチーム開発において、以下の工程の欠落はプロジェクトの頓挫に直結する。

1. 要件定義 (Requirements Definition)

作成するアプリケーションが「誰の」「どのような課題を解決するか」を定義し、実装すべき機能を明確化する工程。

高校生のチーム開発における要件定義の注意点

- 目的の抽象化を避ける

「面白いゲーム」ではなく、「1プレイ3分で完結し、乱数を用いたパズル」など、客観的に評価可能な要件に落とし込む。

- 非機能要件の考慮

「正常に動くか」だけでなく、「計算処理に時間がかかりすぎないか」「操作方法は直感的か」といった要件も事前に定義する。

2. スコープマネジメント (Scope Management)

開発プロジェクトには厳格な「期限」が存在する。要件を満たすためにどこまでの機能を実装するか（スコープ）を管理・制限する必要がある。

MVP (Minimum Viable Product) の概念

- 初期段階から全機能の完成を目指すのではなく、「要件を満たす**最小限の機能（MVP）**」を最優先で実装する。
- 例：「敵のAI」「豪華なエフェクト」は後回しにし、まずは「自機が動いて弾が撃てる」中核モジュールを期限内に完成させる。
- 必須機能（Must-have）と付加機能（Nice-to-have）の厳密な切り分けが、時間切れによる未完成を防ぐ。

3. モジュール分割とインターフェース設計

システムを複数の小さな部品（モジュール）に分割し、分業を可能にする工程。

ブラックボックス化とインターフェース

- モジュールの内部構造（どのようなアルゴリズムで計算するか）は、他の担当者が知る必要はない（ブラックボックス化）。
- 重要なのは、**モジュール間のインターフェース（入出力の仕様）**を事前に厳密に取り決めることである。
- 例：関数 `calculate_score()` は、引数として「整数値の配列」を受け取り、戻り値として「0 から 100 の浮動小数点数」を返す、という規約をチーム内で共有する。

4. タスクの依存関係と並行開発

作業を細分化（WBS: Work Breakdown Structure）したのち、タスク間の依存関係を整理する。

「待ち時間」の排除（モックの活用）

- 依存関係の罫：「Aさんのデータ処理ロジックが完成するまで、Bさんの画面描画ロジックのテストができない」。
- この状態を防ぐため、インターフェース仕様に基づいた**ダミーデータ（モック）**を活用する。
- Aさんのモジュールが未完成でも、Bさんは「仕様通りに仮の配列を出力するモック関数」を呼び出すことで、自身のモジュール開発を独立して進行できる。

5. 構成管理 (Configuration Management)

複数の開発者が書いたコードを統合し、バージョンを管理する工程。

ファイルの競合と先祖返り

- 各自が手元の PC で同名ファイルを編集し、単一の共有フォルダに上書きすると、他者の実装したコードを消去する「競合」が発生する。
- 変更履歴が追えなくなり、`main_v2_final_最終版.py` のような命名規則の崩壊を招く。
- ファイルの命名規則、更新日時の記録、関数単位での結合テストのルール等、コードを統合する際の手順をプロジェクト初期段階で制定することが求められる。

次週に向けた準備

- 月曜の授業でチーム分け
- コードやアプリの精度はそこまで評価として大きくない
- 要件定義や設計図など文章部分が多めの配点
- 期末試験はこちらは少なめ
- コーディングのみ AI 使用可能
- 要件定義と完成品への FB で評価
- 来週はとりあえずオブジェクティブ指向のコーディングを扱う